From C++ Source to Machine Instructions

```
std::cout << "Hello, world!\n";
```

- C++ source is written for people to read and maintain.
- A compiler translates it into instructions for a target processor.
- The processor executes machine-code instructions; assembly is a readable view of instructions.
- The exact instructions depend on the compiler and processor.

A Comparison Becomes a Branch

```
if (left > right) {
    std::cout << left;
} else {
    std::cout << right;
}
```

One possible *conceptual* instruction sequence:

```
COMPARE left, right
JUMP-IF-NOT-GREATER to_else
PRINT left
JUMP to_end
to_else: PRINT right
to_end:
```

- This is pseudocode, not real assembly syntax; processors differ.

Follow the Branch

```
if (score >= 60) {
    std::cout << "Pass\n";
} else {
    std::cout << "Try again\n";
}
```

`score`	Test `score >= 60`	Path taken	Output
`75`	true	first branch	`Pass`
`40`	false	`else` branch	`Try again`

- The condition changes which instruction runs next.

Repetition in Everyday Tasks

- A repeated task has an action that happens again until a stopping point.
- Examples: read pages until the end, practice until time is up, or process each entry.
- Programs use loops to repeat instructions without copying the same code many times.
- A loop needs a condition that can eventually become false.

Repeat with `while`

Print `Hello` three times without copying the output statement.

```
int count = 0;

while (count < 3) {
    cout << "Hello\n";
    count++;
}
```

- Test `count < 3` before each pass through the body.
- Increase `count` so the loop can eventually stop.

Trace a Loop

For `while (count < 3)`, the condition is tested before the body runs.

`count` at test	Condition	Action	`count` after update
`0`	true	print `Hello`	`1`
`1`	true	print `Hello`	`2`
`2`	true	print `Hello`	`3`
`3`	false	stop	`3`

- The body runs three times, not four.

A Loop Needs Progress

```
int remaining = 3;
while (remaining > 0) {
    cout << remaining << '\n';
    remaining--;
}
```

1. Initialize the value used by the condition.
2. Test the condition.
3. Run the body and update the value.
4. Test again; stop when the condition is false.

- If `remaining--` is missing, the value stays `3` and the loop never ends.

PRACTICE

Assignment: Guess the Number

- Choose a secret integer from `0` through `9` (start with a fixed value such as `7`).
- Ask the player to guess; repeat while the guess is not correct.
- After each wrong guess, say whether the secret is higher or lower.
- Count attempts and print the number of guesses when the player wins.

Test: Guess `3`, then `8`, then `7` when the secret is `7`; the game should stop after the third guess.

- Extension: generate a different secret each run.

Challenge: Decimal Integer to Binary

- Read a non-negative integer and print its base-2 representation.
- Special case: `0` converts to `0`.
- Repeatedly divide by `2` and record each remainder.
- Read the remainders in reverse order to get the binary digits.

Decimal	Remainders in discovery order	Binary
`13`	`1, 0, 1, 1`	`1101`
`8`	`0, 0, 0, 1`	`1000`

- The first remainder is the rightmost binary digit.
- Test `0`, `1`, `7`, `8`, `12`, and `15`.